

Assistant Agentic RAG

Assistant pédagogique de questions-réponses construit manuellement avec LangGraph, sans `create_agent`, combinant recherche vectorielle, routage agentique, outils spécialisés et vérification de *groundedness*.

ÉTUDIANT

Marwane Qada

DÉPÔT GITHUB

github.com/marwaneqada/bigdata-agentic-rag

1 Objectif et corpus documentaire

L'objectif du projet est de construire un assistant capable de répondre à des questions de cours sur le Big Data et le Cloud à partir d'un corpus local. Le système combine recherche vectorielle, routage agentique, outils spécialisés, génération contrôlée et vérification de *groundedness*.

Le corpus indexé après nettoyage contient **6 documents de cours** et **18 chunks** :

Apache Airflow

Hadoop HDFS

MinIO

Kafka Streams

Spark SQL

Spark Structured Streaming

Les fichiers de métadonnées et de documentation projet (`README` , `SOURCES.md` , `A_LIRE_AVANT_ENVOI.txt`) sont exclus de l'indexation afin d'éviter qu'ils apparaissent comme sources de réponse.

2 Prétraitement, découpage et vectorisation

Le module `src/document_loader.py` découvre les fichiers PDF, Markdown et texte, nettoie le contenu, puis le découpe en chunks par paragraphes avec une taille cible de 900 caractères et un chevauchement de 150 caractères. Les chunks trop courts sont ignorés.

Les embeddings sont produits avec `paraphrase-multilingual-MiniLM-L12-v2`, puis stockés dans une base **Chroma** persistante. L'index est reconstruit avec `python ingest.py --reset`. La dernière indexation validée a produit **18 chunks** à partir des **6 documents** de cours.

6DOCUMENTS DE
COURS**18**

CHUNKS INDEXÉS

ChromaBASE VECTORIELLE
Persistante sur disque**MiniLM-L12**EMBEDDINGS
Multilingues

CHAÎNE D'INDEXATION

DécouvertePDF · Markdown ·
texte**Nettoyage**

contenu filtré

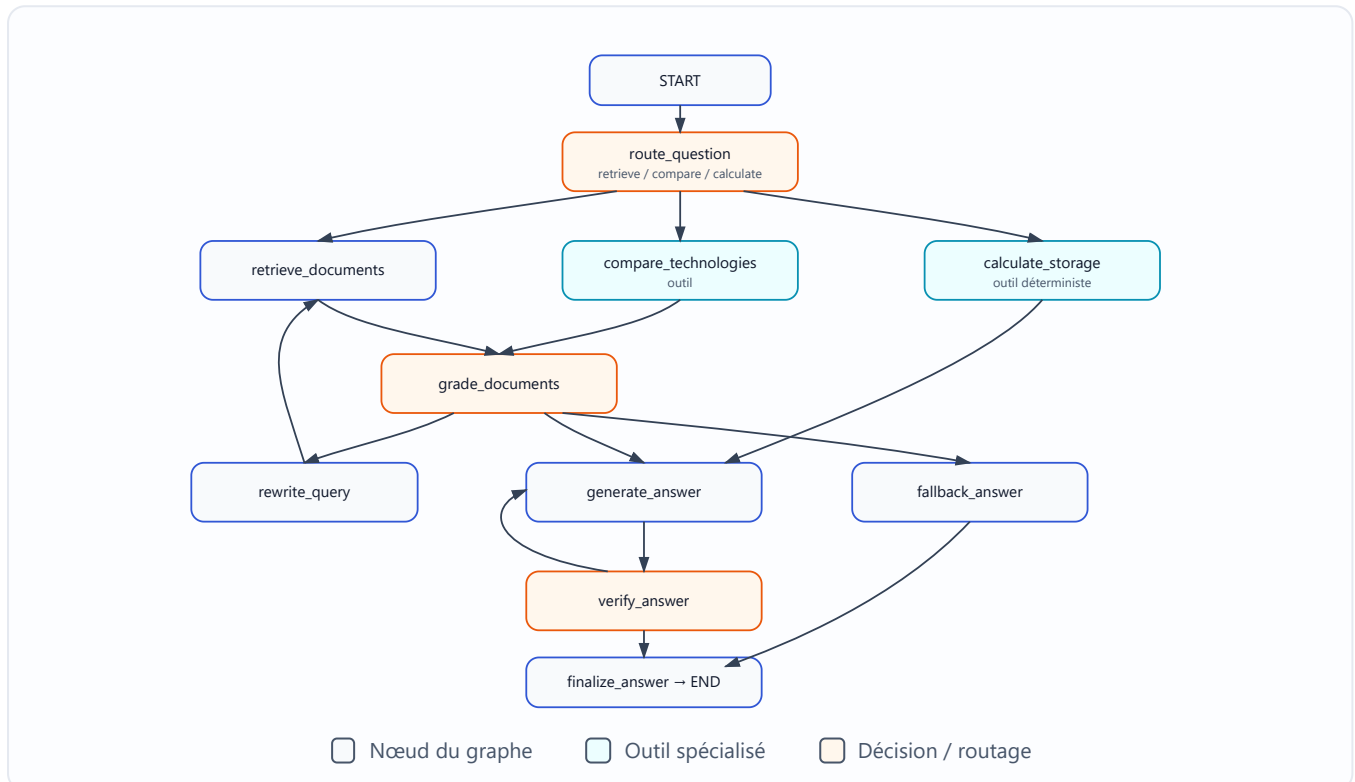
Découpage900 c · chevauch. 150
c**Embeddings**MiniLM-L12
multilingue**Index Chroma**

18 chunks · 6 docs

3 Architecture Agentic RAG avec LangGraph

L'architecture est définie dans `src/graph.py` à l'aide d'un `StateGraph` construit manuellement. Le graphe contient les nœuds `route_question`, `retrieve_documents`, `compare_technologies`, `calculate_storage`, `grade_documents`, `rewrite_query`, `generate_answer`, `verify_answer`, `fallback_answer` et `finalize_answer`.

Le flux général est le suivant : classification de la question, sélection de route, récupération ou outil spécialisé, filtrage de pertinence, génération, vérification, puis finalisation. Si les documents ne sont pas assez pertinents, la requête peut être reformulée avant une nouvelle récupération.



4 Routage, outils, mémoire et vérification

Routage et outils. Le nœud `route_question` classe la question vers l'une des trois routes (détaillées en page 3). Les outils principaux sont définis dans `src/tools.py` : recherche documentaire dans Chroma, comparaison de technologies avec récupération de preuves pour chaque terme comparé, et calcul local déterministe de capacité.

Mémoire conversationnelle. Elle est assurée par un `checkpoint` en mémoire et un `thread_id`. Suffisante pour une démonstration locale, elle n'est pas persistante après redémarrage.

Filtrage et reformulation. Le nœud `grade_documents` filtre les chunks individuellement afin de ne conserver que les extraits pertinents. Pour les comparaisons, la diversité des sources est préservée pour garder des preuves sur les deux technologies. Si la pertinence est insuffisante, `rewrite_query` reformule la requête avant une nouvelle récupération.

Vérification. La vérification de *groundedness* est effectuée avant la finalisation. Ses informations restent dans les détails d'exécution et ne sont pas ajoutées comme commentaires visibles dans la réponse finale.

5 Méthodologie et résultats d'évaluation

L'évaluation finale a été exécutée avec `python -m evaluation.run_evaluation` sur **20 questions (10 simples et 10 complexes)**. Les résultats ont été exportés vers `results/evaluation_results.csv` et `results/evaluation_summary.json`.



Indicateurs validés — `evaluation_summary.json`

INDICATEUR	RÉSULTAT
Questions totales	20
Questions simples	10
Questions complexes	10
Temps moyen de réponse	32,426 s
Qualité (auto.)	5,0/5
Pertinence documentaire (auto.)	5,0/5
Groundedness (auto.)	5,0/5
Erreurs	0
Réponses vides	0
Fallbacks	0
Tests automatisés	5 passés

Les trois routes du système. Le routage dirige chaque question vers le comportement attendu.

● retrieve

Questions standards de cours : définitions, explications et commandes.

● compare

Comparaisons explicites entre technologies ou concepts.

● calculate

Calculs numériques de stockage, réplication et compression.

$500 \text{ GB} \times 0,6 \text{ compression} \times \text{réplication } 3 \times 2 \text{ copies}$
= **1800 GB**

Évaluation automatisée. Les scores de qualité, de pertinence et de *groundedness* (5,0/5) sont attribués par un évaluateur LLM automatisé. Ils ne constituent donc pas une évaluation humaine indépendante et peuvent être génériques ; ils doivent être interprétés comme un contrôle interne et non comme une validation externe.

6 Résultats principaux

Les routes validées couvrent les trois comportements attendus : recherche documentaire, comparaison et calcul. Les questions de comparaison utilisent la route `compare`, la question **C06** utilise `calculate`, et les questions standards utilisent `retrieve`. Les sources affichées ne contiennent plus de fichiers de métadonnées.

0

ERREURS

0

RÉPONSES VIDES

0

FALLBACKS

5

TESTS PASSÉS

7 Limites et pistes d'amélioration

Limites principales

- corpus réduit à six documents ;
- latence moyenne d'environ 32 secondes ;
- plusieurs appels LLM par question ;
- état conversationnel en mémoire uniquement ;
- dépendance à Groq et à l'accès Internet ;
- évaluation automatisée potentiellement génèreuse.

Pistes d'amélioration

- élargir le corpus de documents de cours ;
- réduire la latence et le nombre d'appels LLM par question ;
- rendre l'état conversationnel persistant après redémarrage ;
- compléter l'évaluation automatique par une évaluation humaine indépendante ;
- limiter la dépendance à un fournisseur LLM et à l'accès Internet.

Conclusion

Le projet fournit un assistant Agentic RAG fonctionnel pour un corpus pédagogique Big Data et Cloud. L'architecture LangGraph reste explicite et maintenable, les outils locaux complètent la génération LLM, et l'évaluation finale confirme que les **20 questions** ont été traitées sans erreur, sans fallback et sans réponse vide. Le système est **prêt pour la présentation académique**, sous réserve de mentionner clairement les limites liées au corpus, à la latence et à l'évaluation automatique.